# Study Buddy Enhanced

LSDTFH[1]

University of the Witwatersrand

August 5, 2025 – October 17, 2025

| Project Team | |
|---|---|
| **Author Name** | Student Number |
| **Lebo Kharafu** | 2577390 |
| **TrustGod Mokgwadi** | 2705185 |
| **Favour Mtshali** | 2662283 |
| **Dimpho Matea** | 2678460 |
| **Sinethemba Khumalo** | 2672254 |
| **Hulisani Netshaulu** | 2769364 |

# Contents

# Chapter 1

# Project Road-map

## 1.1 Hidden Requirements

| **Filter by courses** |
| --- |
| **Send notifications** |
| **Link similar courses** |

## 1.2 Development Phases

The project follows four iterative sprints aligned with course requirements:

### 1.2.1 Sprint 1 (Foundation)

- **Focus**: System setup and core infrastructure
- **Key Deliverables**:
  - Authentication system (Google OAuth)
  - Version control implementation
  - Documentation site (VitePress)
  - Project management framework (Crystal Clear)
- **Rubric Weight**: 80% of sprint grade

### 1.2.2 Sprint 2 (Core Features)

- **Focus**: Implementation of primary functionality
- **Key Deliverables**:
  - Course specification system
  - Buddy matching algorithm

  - Resource sharing with PDF tagging
  - Google Calendar integration

- **Rubric Weight**: 95% of sprint grade

### 1.2.3 Sprint 3 (Feature Completion)

- **Focus**: Enhanced functionality and refinement
- **Key Deliverables**:
  - Discussion threads with Markdown
  - Similar course tagging
  - Progress tracking dashboard
  - Accessibility improvements

### 1.2.4 Sprint 4 (Polishing)

- **Focus**: Final touches and documentation
- **Key Deliverables**:
  - Comprehensive testing coverage
  - Group and individual reports
  - Final presentation materials
  - Stretch goals (if time permits)

## 1.3 Feature Prioritization

Table 1.1: Feature Priority by Sprint

| **Priority** | **Sprint** | **Key Features** |
|---|---|---|
| Critical | 1 | Auth, Docs, Setup |
| High | 2 | Courses, Matching, Resources |
| Medium | 3 | Discussions, Progress |
| Low | 4 | Offline support |

## 1.4 Technical Implementation

### 1.4.1 Core Stack

- **Frontend**: React with TypeScript

- **Backend**: Node.js/Express
- **Database**: PostgreSQL (NeonDB) + Sequalise
- **Testing**: Cypress + Vitest

### 1.4.2 Integration Points

- Google OAuth for authentication
- Google Calendar API for scheduling
- Socket.IO for real-time chat
- Firebase for notifications

## 1.5 Risk Management

- **Technical Risks**: API rate limits, database scaling
- **Schedule Risks**: Feature creep in Sprint 2
- **Mitigation Strategies**:
  - Weekly client check-ins
  - Automated testing coverage
  - Priority-based backlog grooming

# Chapter 2

# Project Management

## 2.1 Overview

Crystal Clear is the simplest and lightest member of the Crystal Methods family, designed for small teams (typically 16 people) working on low-criticality software projects, such as web applications or prototypes. It emphasizes people, communication, and minimal overhead to deliver working software quickly. This methodology is particularly well-suited for academic or small-scale commercial projects where flexibility and rapid iteration are valued over rigorous, heavyweight processes. By focusing on core agile properties and lightweight practices, Crystal Clear enables teams to maintain productivity and adapt to changes efficiently.

## 2.2 Methodology

Crystal Clear is built upon seven core properties that ensure project success:

1. **Frequent Delivery**: Deliver working software increments every 13 months (or more frequently for smaller features) to gather user feedback.
2. **Reflective Improvement**: Hold regular retrospectives to assess processes and implement improvements.
3. **Close Communication**: Encourage frequent and direct communication among team members, whether in-person or via digital means.
4. **Personal Safety**: Foster an environment where team members feel safe to express ideas and concerns without fear of blame.
5. **Focus**: Ensure team members have clear tasks and dedicated time to complete them without interruptions.
6. **Easy Access to Users**: Maintain regular contact with end-users to validate features and usability.

7. **Technical Environment**: Use supportive tools like version control, automated testing, and continuous integration.

## Key Roles

- **Sponsor**: Provides funding and high-level direction.
- **Senior Designer-Programmer**: Leads technical decisions and mentors others.
- **Designer-Programmers**: Develop features and implement solutions.
- **Business Experts/Users**: Offer domain knowledge and end-user feedback.

## Key Practices

Lightweight practices include:

- Frequent delivery of usable code.
- Reflective improvement via retrospectives.
- Osmotic communication through co-location or digital proximity.
- Use of task boards (e.g., Trello) for priority management.
- Regular user feedback sessions.
- Automated testing and version control.

## Deliverables

Essential work products include:

- Release sequence (high-level feature timeline)
- User stories/use cases
- Risk list (prioritized risks and mitigations)
- Iteration plan (short-term task listing)
- Design notes (key technical decisions)
- Working software (primary output)
- Test cases (automated validation)

## 2.3 Why Crystal Clear is Recommended

Crystal Clear is ideally suited for small teams developing low-criticality software, such as web applications or prototypes, for the following reasons:

- **Small Team Focus**: Designed for teams of 16 people, making it scalable for academic or startup environments.
- **Low Criticality**: Suitable for projects where failures do not result in significant harm, such as educational tools or game prototypes.
- **Flexibility**: Allows teams to prioritize coding and collaboration over bureaucratic processes, aligning well with iterative development common in web projects using HTML, JavaScript, or Python.
- **Beginner-Friendly**: Easy to adopt, even for teams new to agile methodologies, due to its minimalistic and intuitive practices.
- **Emphasis on Communication**: Promotes frequent and open dialogue, reducing misunderstandings and accelerating problem-solving.

## 2.4 Industry Resources & Justification

- Agile Alliance Glossary - Crystal: The Agile Alliance, a premier global agile community, provides an overview of Crystal, affirming its status as a recognized and legitimate agile methodology.
- ScienceDirect Topic on Crystal Method: An academic resource that discusses Crystal's place in software engineering, highlighting its human-powered and adaptive nature, which justifies its recommendation for small teams.
- Agile Business Consortium - Crystal Clear: Provides a concise breakdown of the methodology's key elements, supporting the practical steps for implementation outlined in this chapter.
- VersionOne Agile 101 - Crystal Method: VersionOne (now part of CollabNet VersionOne) was a leading vendor in agile lifecycle management tools. Their guide reinforces the suitability of Crystal for small, co-located teams working on low-criticality projects.

# Chapter 3

# Technology Stack

## 3.1 Frontend Stack

The frontend of the system is designed to provide a responsive, interactive, and user-friendly interface. It emphasizes modularity, reusability, and maintainability of components while ensuring fast load times and smooth navigation.

### 3.1.1 Framework and Libraries

- **React.js** — Used as the primary framework for building a component-based user interface. React enables efficient rendering through its virtual DOM and promotes reusability of UI components.
- **Vite** — Adopted as the build tool and development server. Vite provides a faster development experience with hot module replacement and optimized builds.
- **CSS** — Utilized for styling to achieve a clean, modern, and responsive design with utility-first classes.
- **Lucide Icons** — Used for lightweight, customizable icons across the interface.

### 3.1.2 Design Principles

- **Responsiveness** — Ensures seamless user experiences across devices with different screen sizes.
- **Accessibility (aria)** — Follows best practices to make the application usable for all users, including those with disabilities.
- **Performance Optimization** — Lazy loading, code splitting, and optimized assets improve loading times.

### 3.1.3 Testing and Quality Assurance

- **Playwright** — Applied for integration testing of whole app to guarantee reliability.

### 3.1.4 Deployment

- **Vercel** — The frontend is deployed on Vercel, leveraging its CDN for fast global delivery and integration with GitHub for continuous deployment.

### 3.1.5 Industry Resources

To justify the frontend technology choices, several authoritative industry resources were consulted:

- **JavaScript (JS)**: Recognized as the most widely used programming language for web development, according to the Stack Overflow Developer Survey. `https://survey.stackoverflow.co/2024/`
- We could have used other technologies such as Python,Dart and TypeScript
- **Advantages of JS** :- universal browser support ,runs on all major browsers. Massive Ecosystem Libraries & frameworks (React, Vue, Angular, Node.js, Express) are mature.
- **Disadvantages of comparable technologies**:- Python Slow performance in browsers, limited support for frontend interactivity, extra layers to run in browser.TypeScript Needs compilation step, slightly steeper learning curve. `https://dev.to/codingcatdev/typescript-is-freaking-hard-20k6#:~:text=The%20primary%20challenge%20in%20learning%20TypeScript%20lies%20in,the%20strict%20type%20requirements%20of%20TypeScript%20initially%20overwhelming.`
- **React**: Maintained by Meta and widely adopted in industry, React is considered a standard for building scalable, component-based user interfaces. `https://react.dev/`
- Chosen as majority of group is comfortable with this framework.
- **Advantages of React** :- Component-Based Architecture Build reusable UI components, making code more modular.Learning Curve Easier to get started with compared to Angular or Vue (once you know JS)
- **Disadvantages of comparable technologies**:- Vue.js ,smaller ecosystem, not as many large-scale apps as React, less corporate backing.Angular -has a steeper learining curve with heavier framework and verbose code and is written in typescript which is difficult. `https://kinsta.com/blog/angular-vs-react/#angular-vs-react-an-indepth-comparison`

- **Vite**: A next-generation frontend tooling that provides fast Hot Module Replacement (HMR) and optimized builds, recommended by many open-source contributors and adopted across projects for its developer experience.
- Chosen as majority of group is comfortable and familiar with this.
- **Advantages of Vite** :- Framework-Agnostic Works with React, Vue, Svelte, Preact, Solid, etc.Hot Module Replacement (HMR) Changes appear in the browser almost instantly.
- **Disadvantages of comparable technologies**:-Webpack,slower dev server, complex configuration.Parcel,smaller community and not as broad as vite `https://betterstack.com/community/guides/scaling-nodejs/vite-vs-webpack/`
- **CSS**: CSS is simple to use, and what was familiar to the group. No hassle, no need to learn syntax making the whole design experience seamless.
- Chosen as majority of group is comfortable and familiar with this.
- **Advantages of CSS** :- Universal Support Works in all browsers without any preprocessing or plugins. and it allows full control.
- **Disadvantages of comparable technologies**:Sass/SCSS,Requires compilation, adds complexity.Tailwind CSS,can create messy HTML with many classes; learning curve for utility-first mindset. `https://dev.to/bridget_amana/the-sass-vs-css-debate-is-sass-really-that-superior-277g`

## 3.2 Backend Stack

Overview: This document provides a comprehensive guide to setting up and running the SDP Project backend API. The backend is built with Node.js, Express, Sequelize ORM, and PostgreSQL (NeonDB).

**TECHNOLOGY STACK FOR BACKEND**

- **Runtime:** Node.js
- **Framework:** Express.js
- **Database:** PostgreSQL (NeonDB)
- **ORM:** Sequelize
- **Documentation:** Swagger
- **Authentication:** JWT (JSON Web Tokens)
- **Development Tool:** Nodemon

**PREREQUISITES**

- **Node.js:** v16 or higher
- **Package Manager:** npm
- **Database:** PostgreSQL (NeonDB recommended)
- **Version Control:** Git

### 3.2.1 Database System

**Option A: Local PostgreSQL**

- **Install PostgreSQL locally**
- **Create a database**
- **Update `.env` with local credentials**

**Option B: NeonDB (Recommended)**

- **Sign up at `https://neon.tech`**
- **Create a new project**
- **Copy the connection string**
- **Update `.env` with NeonDB credentials**

### 3.2.2 Industry Resources

**BACKEND References used:**

- **Node.js**: Node.js is a JavaScript runtime built on Chrome's V8 engine. It allows the development of fast and scalable network applications using JavaScript on the server side.
- **Advantages of Node.js**: Event-driven, non-blocking I/O makes it lightweight and efficient; same language (JavaScript) on both client and server speeds up development.
- **Disadvantages of comparable technologies**: Django (Python-based) Steeper learning curve for JS developers and heavier runtime. Ruby on Rails Convention-heavy, slower performance under high concurrency. `https://www.geeksforgeeks.org/blogs/nodejs-vs-django/`
- **Express.js**: Express is a fast, minimalist web framework for Node.js used to build APIs and web applications.
- **Advantages of Express.js**: Minimal setup, middleware support, high flexibility and a large ecosystem of plugins.

- **Disadvantages of comparable technologies**: Koa.js More modular but lacks built-in routing and middleware. Fastify Faster in some benchmarks but smaller community and ecosystem. `https://www.geeksforgeeks.org/blogs/nodejs-vs-django/`
- **PostgreSQL**: PostgreSQL is a powerful, open-source relational database known for its reliability and feature set.
- **Advantages of PostgreSQL**: ACID compliance, strong support for JSON, advanced indexing and extensibility for complex applications.
- **Disadvantages of comparable technologies**: MySQL Faster for simple reads but weaker at advanced queries and standards compliance. MongoDB Flexible document store but no strict relational schema or ACID guarantees by default. `https://www.geeksforgeeks.org/postgresql/difference-between-postgresql-and-mongodb/`
- **Swagger**: Swagger (OpenAPI) is a toolset for designing, building, documenting, and consuming RESTful web services.
- **Advantages of Swagger**: Automatically generates interactive API documentation, enforces API standards, improves team collaboration.
- **Disadvantages of comparable technologies**: Postman Great for testing but weaker at automatic API documentation. RAML Similar to OpenAPI but less widely adopted. `https://apidog.com/articles/postman-vs-swagger/`
- **NeonDB**: NeonDB is a serverless PostgreSQL platform designed for high scalability and developer-friendly workflows.
- **Advantages of NeonDB**: Automatic scaling, built-in branching for databases, zero-downtime operations, and fully managed infrastructure.
- **Disadvantages of comparable technologies**: Supabase Offers a Postgres backend with built-in auth but less fine-grained control. Amazon RDS Powerful and flexible but more complex to configure and manage. `https://www.bytebase.com/blog/neon-vs-supabase/`

## 3.3 Testing Approach

[Testing philosophy and coverage goals]

We aim to best the APIs using vitest because it fast and simple. while for frontend e2e testing os more reliable. I covers most cases and most integration.

### 3.3.1 Unit Testing

[Testing tools and methodology] Backend - vitest

### 3.3.2 End-to-End Testing

[UI automation strategy] Cypress - frontend testing Replaced by Playwright due to the multi window feature

### 3.3.3 Industry Resources

[Testing ROI studies or standards] `https://testgrid.io/blog/cypress-testing/` `https://www.testim.io/blog/cypress-vs-selenium/` `https://dev.to/walmyrlimaesilv/10-reasons-why-you-should-use-cypress-for-web-testing-automation-jlb`

- **Vitest**: Vitest is a fast, Vite-native unit testing framework designed for modern frontend applications.
- **Advantages of Vitest**: Blazing fast test runs thanks to Vite integration, first-class TypeScript support, watch mode for instant feedback, and easy migration from Jest.
- **Disadvantages of comparable technologies**: Jest More mature and widely used but slower on large codebases. Mocha Flexible but requires additional libraries for assertions and mocking. `https://vitest.dev/`
- **Cypress**: Cypress is an end-to-end (E2E) testing framework for web applications, running directly in the browser for real-world testing.
- **Advantages of Cypress**: Fast and reliable E2E tests, real-time reloading, automatic waiting for elements, powerful debugging tools, and a vibrant community.
- **Disadvantages of comparable technologies**: Playwright Cross-browser and powerful but newer with less built-in GUI support. Selenium Very flexible and language-agnostic but slower setup and execution compared to Cypress. `https://www.cypress.io/`

## 3.4 Development Tools

[Team development workflow]

### 3.4.1 Version Control

[Git workflow and branching model]

### 3.4.2 CI/CD Pipeline

[Automation tools and stages]

### 3.4.3 Hosting Platform

**Backend hosted on Render**

### 3.4.4 Industry Resources

[Reliability engineering standards] https://www.conventionalcommits.org/en/v1.0.0/ https://blog.devgenius.io/make-a-meaningful-git-commit-message-with-semantic-commit-me

# Chapter 4

# Git Methodology

## 4.1 Overview

Git is a distributed version control system designed to handle everything from small to very large projects with speed and efficiency. It allows multiple developers to work on the same codebase simultaneously without interfering with each other's work. This chapter outlines the Git strategies, branching models, and versioning conventions recommended for your project. By adopting a consistent methodology, your team can improve collaboration, track changes effectively, and maintain a stable codebase throughout the development lifecycle.

## 4.2 Methodology

A well-defined Git workflow is essential for team coordination and code management. The following practices are recommended:

### Branching Strategy

- **Main Branch**: The `main` branch contains production-ready code. It should always be stable and deployable.
- **Development Branch**: The `develop` branch serves as an integration branch for features. It is used to merge completed features before they are moved to `main`.
- **Feature Branches**: For each new feature, create a branch from `develop` named `feature/<feature-name>`. This keeps work isolated until the feature is complete and tested.
- **Release Branches**: When preparing a release, create a `release/<version>` branch from `develop`. This allows for final testing and bug fixes without disrupting ongoing development.

- **Hotfix Branches**: For urgent fixes in production, create a `hotfix/<issue>` branch from `main`. Once fixed, merge it into both `main` and `develop`.

### Versioning Convention

For this project, a Calendar Versioning (CalVer) approach is recommended due to its simplicity and relevance for tools with frequent iterations:

- **Format**: `YYYY.MM.DD_MODIFIER`
- **Example**: `2025.08.12_sprint1`
- This format provides clarity on the timeline and context of the release (e.g., which sprint it corresponds to).

### Commit Practices

- Write clear, descriptive commit messages.
- Use the imperative mood (e.g., "Add login feature" instead of "Added login feature").
- Commit often to ensure changes are incremental and traceable.

## 4.3 Why This Git Approach is Recommended

This Git methodology is ideal for your team and project for the following reasons:

- **Simplicity**: The branching strategy is easy to understand and implement, even for teams new to Git.
- **Flexibility**: Feature branches allow developers to work independently without affecting the main codebase.
- **Traceability**: CalVer provides clear, meaningful version numbers that reflect development cycles.
- **Scalability**: The workflow supports both small and large teams, making it suitable for your current and future projects.
- **Industry Alignment**: This approach is widely used in open-source and commercial projects, ensuring best practices are followed.

## 4.4 Industry Resources & Justification

The proposed Git methodology and versioning convention are supported by industry standards and authoritative resources:

### Git and Version Control

- Official Git Documentation: The definitive guide to Git commands and concepts.
- A Successful Git Branching Model: The classic article introducing Git-Flow, which inspired the branching strategy recommended here.

### Versioning Schemes

- Semantic Versioning (SemVer): A widely adopted scheme for versioning software based on meaningful version numbers.
- Calendar Versioning (CalVer): A versioning scheme based on project release dates, ideal for projects with time-based releases.
- Designing a Version: A practical guide on choosing and designing a versioning scheme, supporting the use of modifiers like `_sprint1`.

# Chapter 5

# Frontend Chapter

Your chapter content goes here.

## 5.1 First Section

### 5.1.1 A Subsection

**Team Members:** Dimpho Matea, Lebo K, Sinethemba K

**Dimpho Mathea**

**Description:** I implemented the API endpoints for CRUD regarding resources,my task was to ensure that when resources are posted the file url as well as the picture url and some other meta data is stored on the Neon DB ,these could not be possible if cloudinary could not be setup as a result of Neon db not being able to store Filesand images directly,I had faced multiple challenges were I had to familiarize myself with how Swagger API documentation works as well as the framework the team chose to adopt with regards to the backend ,the resource API was not working and is currently being monitored so as to provide efficient results when needed by the Frontend team.

**Lebo**

**Description:** I was writing the functions that act as middle ware for the Frontend and backend. However I was also mostly Responsible for the testing of the different parts of the Frontend. I setup and wrote all the test e2e style and also had to choose a framework that suits our application.

**Sinethemba K**

**Description:** Led the design of the web application, defining the visual style and user interface, and implemented layouts and styling using CSS to ensure a responsive and polished frontend. Responsible for wireframes and prototype using Lovable AI. Main challenges included defining the vision for the app and the look and feel for the best visual user experience, which entailed choosing

colors that complement each other without being overly stimulating. Translated images and wireframes into code, making elements responsive, testing responsiveness across multiple viewports, and writing code to accommodate each case.

# Chapter 6

# Backend Chapter

## 6.1 First Section

### 6.1.1 A Subsection

**Team Members:** Favour Mtshali, TrustGod M, Hulisani Innocent Netshaulu

**Favour Mtshali**

**Description:** I implemented the API endpoints for CRUD regarding resources,my task was to ensure that when resources are posted the file url as well as the picture url and some other meta data is stored on the Neon DB ,these could not be possible if cloudinary could not be setup as a result of Neon db not being able to store Filesand images directly,I had faced multiple challenges were I had to familiarize myself with how Swagger API documentation works as well as the framework the team chose to adopt with regards to the backend ,the resource API was not working and is currently being monitored so as to provide efficient results when needed by the Frontend team.I also added the likes endpoints so that users can be able to like a resource as well as a resource thread endpoint that will be used for commenting on a resource these are features that will enhance the anchore feature of the application

**TrustGod**

**Description:** I designed the database schema and handled authentication logic.

**Huli**

**Description:** I implemented the api endpoints for manual authentication, social media groups functionality and as well as the followers functionality(friends). My task was to allow users to authenticate using their emails and passwords as an alternative to using their google accounts, When i was implementing the authentication i had to resort to using the FireBase 3rd party authenticater because i had difficulty in using google for manual authentication, thus altering the original tech stack we planned on using initially.

# Chapter 7

# Testing Methodology

## 7.1 Testing Setup Process

This section details the step-by-step process for the testing Methodology for Study Buddy Enhanced.

## 7.2 Testing Strategy

To test the app we have choosen to do End-to-End testing because it covers more Integration that unit testing. Each unit can work alone does not ensure that it works well in context of the app.

## 7.3 Test Cases

We will be testing the API as they are and also seeing how they hold up against spam calls/requests. We will be doing End-to-End for all the part of the app a real user might use and how they will actually use it.

## 7.4 Continuous Integration

We have setup a auto test system on every pull request and every merge with the main branch on both backend and frontend. This allows the barnch to be protected at all costs.

## 7.5 Industry Resources

[Testing standards] `https://medium.com/@louisyoong/cypress-with-react-e2e-vs-component-testing-e20c2`
`https://bugbug.io/blog/software-testing/unit-testing-vs-end-to-end-testing/`

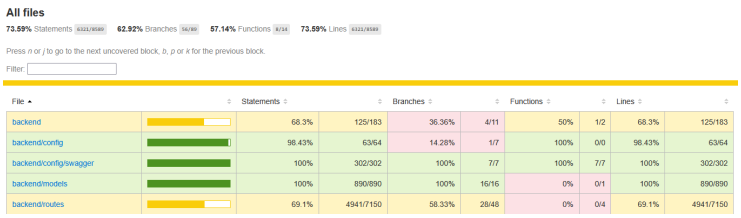
https://momentic.ai/resources/cypress-component-testing-vs-e2e-testing-a-deep-dive-for
https://www.globalapptesting.com/blog/unit-testing-vs-end-to-end-testing

# Chapter 8

# Testing Progress

## 8.1 Backend Tested

### 8.1.1 Coverage



**All files**

**73.59%** Statements 6321/8589 **62.92%** Branches 56/89 **57.14%** Functions 8/14 **73.59%** Lines 6321/8589

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

| File | Statements | | Branches | | Functions | | Lines | |
|---|---|---|---|---|---|---|---|---|
| backend | 68.3% | 125/183 | 36.36% | 4/11 | 50% | 1/2 | 68.3% | 125/183 |
| backend/config | 98.43% | 63/64 | 14.28% | 1/7 | 100% | 0/0 | 98.43% | 63/64 |
| backend/config/swagger | 100% | 302/302 | 100% | 7/7 | 100% | 7/7 | 100% | 302/302 |
| backend/models | 100% | 890/890 | 100% | 16/16 | 0% | 0/1 | 100% | 890/890 |
| backend/routes | 69.1% | 4941/7150 | 58.33% | 28/48 | 0% | 0/4 | 69.1% | 4941/7150 |

### 8.1.2 Test Cases

✓ **Test-Api-Info**

- **Description:** Information about our Api shows.
- **Reason:** To easily check if our Api is up.

✓ **Test-Public-Course-Get-Endpoint**

- **Description:** Get the all courses from public endpoint.
- **Reason:** Have public endpoint for external use.

✓ **Test-Private-Course-Get-Endpoint**

- **Description:** Get the all courses from private endpoint.
- **Reason:** Have private endpoint for getting courses.

✓ **Test-Private-Resources-Get-Endpoint**

- **Description:** Get the all resources from private endpoint.

  - **Reason:** Have private endpoint for getting resources or specific resource.

✓ **Test-User-Get-Endpoint**

  - **Description:** Check if we can get users by Id or All.
  - **Reason:** To get all users or specific users.

✓ **Test-User-Get-notification**

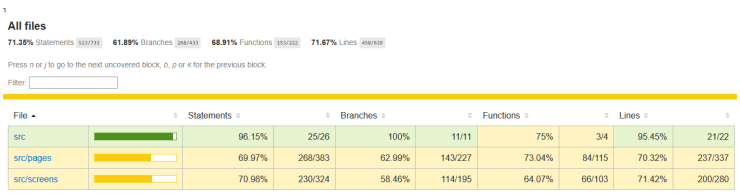
  - **Description:** Check if we can get users by Id or All.
  - **Reason:** To get all users or specific users.

— **Authentication**

  - **Description:** Testing the authentication endpoint directly.
  - **Reason:** The test is not done because it is indirectly tested via login functionality

## 8.2 Frontend Tested

### 8.2.1 Coverage




### 8.2.2 Test Cases

✓ **Test-Login**

  - **Description:** Test if user can correctly with Google email.
  - **Reason:** If user cannot login they cannot use the app.

✓ **Test-Saving-State**

  - **Description:** Saving the Login state for all tests to use
  - **Reason:** To avoid being block for automated testing by Google.

## 8.3 Issues

### 8.3.1 npm Packages 2025-09-21

- 180 packages are looking for funding
- Run `npm fund` for details
- 1 low severity vulnerability
- To address all issues, run: `npm audit fix`
- Run `npm audit` for details

### 8.3.2 Env Files Being Tracked Fix

```
git rm --cached frontend/.env
git rm --cached frontend/.env.*
git rm --cached backend/.env
git rm --cached backend/.env.*
git commit -m "Stop tracking .env files in frontend and backend"
```

## 8.4 Package Audit Report

### 8.4.1 Audit Details

- **Audit Date:** 2025-09-30T13:54:43.105518

### 8.4.2 Frontend Audit

- **Total Packages:** 584
- **Compromised:** 21
- **Compromised Percentage:** 3.6%
- **Safe Percentage:** 96.4%

**Compromised Packages:**

```
- ansi-regex@6.2.2
- ansi-styles@6.2.3
- strip-ansi@7.1.2
- wrap-ansi@8.1.0
- debug@4.1.12
- ansi-regex@5.0.1
- ansi-styles@4.3.0
- chalk@4.1.2
- color-convert@2.0.1
```

- color-name@1.1.4
- debug@4.4.1
- debug@4.3.7
- debug@4.3.7
- debug@1.0.0
- debug@4.3.7
- debug@4.3.7
- debug@4.3.7
- debug@4.3.7
- strip-ansi@6.0.1
- supports-color@7.2.0
- wrap-ansi@6.2.0

### 8.4.3 Backend Audit

- **Total Packages:** 1043
- **Compromised:** 34
- **Compromised Percentage:** 3.26%
- **Safe Percentage:** 96.74%

**Compromised Packages:**

- ansi-regex@6.2.2
- ansi-styles@6.2.3
- strip-ansi@7.1.2
- wrap-ansi@8.1.0
- debug@4.1.12
- ansi-regex@5.0.1
- ansi-styles@4.3.0
- debug@2.6.9
- chalk@4.1.2
- color-convert@2.0.1
- color-name@1.1.4
- debug@2.6.9
- debug@4.4.3
- debug@4.3.7
- debug@4.3.7
- error-ex@1.3.4
- debug@3.2.7
- debug@3.2.7
- debug@3.2.7
- debug@2.6.9
- debug@2.6.9
- is-arrayish@0.2.1
- supports-color@8.1.1

```
- debug@2.6.9
- supports-color@5.5.0
- ansi-styles@5.2.0
- debug@2.6.9
- debug@4.3.7
- debug@4.3.7
- debug@4.3.7
- debug@4.3.7
- strip-ansi@6.0.1
- supports-color@7.2.0
- wrap-ansi@7.0.0
```

### 8.4.4 Overall Summary

- **Total Packages:** 1627
- **Total Compromised:** 55
- **Overall Compromised:** 3.38%
- **Overall Safe:** 96.62%

**Assessment:** The low compromise rate (3.38%) reflects minimal security concerns, as these vulnerabilities originate from external package dependencies rather than our core application code, presenting acceptable risk levels for our current deployment. Hance no action is required.

# Chapter 9

# Meetings Chapter

## 9.1 Sprint One

### 9.1.1 Meeting One - Planning

**Date:** 7 August 2025
**Attendees:** Backend Team
**Discussion:** Roles on the backend and effective communication of the structure.
**Minutes:**

- **Roles Discussion**
  - Discussed the roles that need to be shared among members.
  - Highlighted that one member had too many responsibilities, which hindered effective work distribution and collaboration.
  - **Favour:** RESOURCE tables and file sharing system.
  - **Huli:** Caching of API and group-centric tables.
  - **Trust:** Picture-based files and backend setup.
- **Structure**
  - Emphasized the importance of core files: `models`, `index`, and `server`.
  - Used a platform to visualize the database schema.
- **Tasks and Objectives**
  - Members need to research and become comfortable with the provided tech stack.
  - Proper documentation required for reasoning and comparisons to alternative tools.

### 9.1.2 Meeting Two - Collaboration

**Date:** 13 August 2025

**Attendees:** Backend Team

**Discussion:** Cooperation with the independent developer team and integration of their API service.

**Minutes:**

- **Database Updates & Brainstorming Session**
  - Discussed how to integrate a third-party application that manages events to support the organization of study sessions.
  - Added new database tables to track information relevant to study sessions, specifically for use by group members.

### 9.1.3 Meeting Three - Rubric Review and Coordination

**Date:** 16 August 2025

**Attendees:** Backend & Frontend Teams

**Discussion:** Review of rubric checklist and acknowledgement of successfully completed items.

**Minutes:**

- **Task Review**
  - Agreed on which tasks are considered complete and which remain incomplete.
  - Reviewed the rubric and identified tasks to be implemented before the Sprint 1 deadline.
  - Discussed app components that are not functioning properly.
  - Identified actions to address the issues.
- **Authentication Fault**
  - Backend and frontend authentication failures were raised.
  - Planned further debugging of server-side authentication.
- **API Development Strategy**
  - Agreed that all API routes will be built based on client needs, not developer assumptions.
- **Branch Creation Methodology**
  - Allow individual merges to the version release branch.
  - Only approved merges to the main branch are permitted.
- **LaTeX Documentation**
  - All members are expected to update the shared LaTeX document on Overleaf for each feature they create.

### 9.1.4 Meeting Four - Rubric and LaTeX Documentation

**Date:** 18 August 2025
**Attendees:** Backend & Frontend Teams
**Discussion:** Review of rubric checklist, completed items, and LaTeX documentation.
**Minutes:**

- **LaTeX Platform**
  - Each member logged in and updated the shared LaTeX file.
- **Sprint 1 Rubric Review**
  - Reviewed the rubric and identified required implementations that were missed for Sprint 1.
- **API Requirements from Event Management Group**
  - Groups table will only share `venueID`, `Name`, and `Location`.
  - Created a separate table for the date and time of events.
- **GitHub Committing**
  - Implement semantic committing from now on.

## 9.2 Sprint Two

### 9.2.1 Meeting One - Backend Problem Resolution

**Date:** 31 August 2025
**Attendees:** Backend Team
**Discussion:** Discussion on how the APIS work as there were problems
**Minutes:**

- **API Dissection**
  - Discussed the roles that certain APIs had within building the application.
  - Highlighted the fact that Using swagger to document the API was a challenge ,as a team member that had the responsibilities of a certain API, the API was not working when it was called.
  - **Favour:** The team member with the struggle of making the API work.
  - **Trust:** The Backend facilitator,who help with understanding the pitfalls on the API not working.

### 9.2.2 Meeting Two - Retrospective

**Date:** 1 September 2025

**Attendees:** The whole Team

**Discussion:** Discussion how the sprint went and some resolutions for the following sprints

**Minutes:**

- **API Dissection**
  - Discussed the problematic integration of the API with the frontend as some were not working for days and it hindered progress.
  - Discussed the impact that us not having meetings was a problem as some team members were working blindly and there was no communication that was solid except on whatsApp when a problem arouse we solved it through text and this limited productivity.
  - **Resolutions:** The team members all agreed to try and improve the amount of times we meet and have a fixed schedule for meetings that will not change unless there is an emergency.Smaller meetings to be had with backend team if there is an issue with the backend and not involving the frontend team as this will waste time.Team Discussed the importance of reading messages that are communicated on whatsApp thoroughly as they may contain intel and minimise confusion.
  - **Reasons of Problems:**The team didnt meet as a result of this week being hectic with regards to submissions and there being a test written that needed undivided attention.This truly messed up the teams productivity in balancing and finishing the work in a timely manner.Some days were reserved as some team members have religious practices like the saddath day as well as bible study to uphold,these could have been used to push work thoroughly.

## 9.3 Sprint Three

### 9.3.1 Meeting one - Review

**Date:** 2 September 2025

**Attendees:** Backend Team

**Discussion:** Discussion review as well as Plan of using 2 repo system

**Minutes:**

- **2 repos frontend and backend**
  - The backend team thought it well to actually separate the front end and backend repo .

  - Reasons is to avoid the backend team completely touching the frontend as there was an instance where the front end vite version was changed and it affected everything.
  - **documentation:** Spoke on how us not talking about the documentation properly will mess us up.Continue to send through and write google documents for how users experienced the application.
  - **APIs feedback 3 days prior:**The backend team spoke about the problem of the front end not communicating their needs as there were no frequent meetings.Spoke about there being atleast 3 day prior warning about APISs that are not working accordingly so they could be fixed and avoid unprofessional conduct.

### 9.3.2 Meeting two - review

**Date:** 7 September 2025
**Attendees:** Backend and Frontend Team one member each team
**Discussion:** Fixing PDFs and image uploading.
**Minutes:**

- **Troubleshooting**
  - Spoke about the reason why some APIs are failing and implemented Troubleshooting.
- **UI**
  - It was agreed that the Frontend should be made to be more user friendly.

### 9.3.3 Meeting three - review

**Date:** 11 September 2025
**Attendees:** Backend and Frontend Team
**Discussion:** Syncing the tests with backend procedures.
**Minutes:**

- **APIs and testing sync**
  - Any alterations to the Api endpoints should be alerted to the tester.
  - Before pushing, run npm run test locally, and inspect faults (if any)
- **External API and Public**
  - Resource service is still our main public API.
  - Add an external calendar API, google calendar api spoke to the client and they approved.
  - Add an external API that fetches resources that are already in existence

### 9.3.4 Meeting four - review

**Date:** 22 September 2025
**Attendees:** Backend and Frontend Team
**Discussion:** Rubric, Feature, Requirements and Documentation progress discussion.
**Minutes:**

- **Feature review**
  - The frontend team exhibit their progress on the app .
- **Discussed on the pending endpoints from the backend**
  - Resource feed endpoint.
  - Login streak endpoint.
  - Alter group endpoint payload.
- **Reviewed hidden requirements completions**
  - Filter by course,the user can search question to ask client is it the same as filter by course.
  - Send notifications,by time 30 mins 15 min before session setup but not complete.
  - Link similar courses.
- **Progress with user feedback**
  - We agreed to share the google form to at least 1 person each .
- **More info about API**
  - External API.
  - Authentication API fix.

### 9.3.5 Meeting five - review

**Date:** 26 September 2025
**Attendees:** Backend and Frontend Team 1 member per team
**Discussion:** Documentation consolidation
**Minutes:**

- **Features update documentation**
  - The frontend dev showed the document overseer the features that were implemented.
  - The features as well as the stakeholder feedback chapters updated accordingly.

### 9.3.6 Meeting Six - Retrospective

**Date:** 30 September 2025
**Attendees:** Backend and Frontend Team members
**Discussion:** Team challenges and improvement opportunities
**Minutes:**

- **API Performance**
  - Team members raised concerns about the slowness of the API affecting productivity.
- **Workload and Time Management**
  - The team has many tasks on their plates, leaving limited time to complete planned work effectively.
- **Communication Issues**
  - Messages are often ignored, and important updates are not consistently shared among team members.
- **Maintenance and Bug Fixing**
  - Issues or broken features are not promptly addressed, causing technical debt to accumulate.
- **Testing Practices**
  - Testing is frequently overlooked.
  - Lebo mentioned the possibility of stopping testing due to these ongoing issues.

## 9.4 Sprint Four

### 9.4.1 Meeting One - Planning

### 9.4.2 Meeting Two - Final Retrospective

# Chapter 10

# Pitfall Chapter

Your chapter content goes here.

## 10.1 First Section

### 10.1.1 A Subsection

**Pitfall: Documentation**

**Description:** During development, we documented things but not in order that makes sense and we left out a lot of things that were truly important.

**Resolution:** Each member was instructed that when documenting on Papeeria to ensure that errors are mitigated immediately and not left for other people to solve.

**Pitfall: API connection to Frontend**

**Description:** There were multiple APIs that were built and not working so when the Frontend needed them they found out that they were not working,hence curbing productivity this is also as a result of the Backend having multiple issues.

**Resolution:** Had small meeting with the Backend to ensure smooth running APIs as well as agreed that they should be tested before the Frontend team uses them.

**Pitfall: External API not ready**

**Description:** Initially we had a team building the Event management application that agreed to integrate their API with our team after some days they reached out and informed us that they cannot provide the API we need so that we can integrate our application.

**Pitfall: Frontend issues**

Frontend does not know how to utilise backend APIs, beacuse they don't understand how to Backend is structured. which is something backend team offered to give lessons on, but the frontend team did not not reach out.

**Pitfall: Frontend incompetency**

Frontend team is always deliverying late, causing the backend team to have a short period of time to fix their apis.

## Pitfall: Frontend and Backend syncing issue

Frontend usually require changes to be made to backend apis, into the format the think it seems fit. This leads to backend making changes to their apis, database tables and investigating if the other apis will still work as they used to. Furthermore hindering progress in the backend.

**Resolution:** The team will continue to look for alternative teams that can integrate our API with our application or change the way in which we need the API incase we do not in a timely manner.

# Chapter 11

# User Feedback

## 11.1 Overview

This chapter presents feedback from users who interacted with Study Buddy. It highlights the users, experiences, perceptions, and suggestions. The goal is to understand how the application performs in real scenarios, what users like, and areas for improvement.

## 11.2 Purpose of the Chapter

The purpose of this chapter is to:

- Summarize user experiences.
- Identify strengths and weaknesses of Study Buddy.
- Provide actionable insights for future improvements.

## 11.3 User Feedback Responses

### User 1: Brilliant Mukgwedi

- Likely usage: Daily
- Main purpose: Accessing or sharing resources
- Rating: 2 stars
- Ease of navigation: Difficult
- Most useful feature: Resource sharing
- Satisfaction with group discussions and private messages: Not satisfied

- Comments: Likes the website for educational purposes and obtaining resources for courses. Suggested adding more features to make it holistic.

### User 2: Lufuno Tshifhiwa

- Likely usage: Several times a week
- Main purpose: Accessing or sharing resources
- Rating: 5 stars
- Ease of navigation: Very easy
- Most useful features: Resource sharing, connecting with new people
- Satisfaction with group discussions and private messages: Very satisfied
- Comments: Likes the ease of use of the website.

### User 3: Devine Onwenu

- Likely usage: Occasionally
- Main purpose: Accessing or sharing resources
- Rating: 3 stars
- Ease of navigation: Easy
- Most useful feature: Connecting with new people
- Satisfaction with group discussions and private messages: Neutral
- Comments: Likes the simplicity of the website and suggested improvements for accessibility.

### User 4: Maybe Kgaugelo

- Likely usage: Occasionally
- Main purpose: Group discussions
- Rating: 5 stars
- Ease of navigation: Easy
- Most useful feature: Group discussions
- Satisfaction with group discussions and private messages: Satisfied
- Comments: Enjoys group discussions with friends.

### User 5: Malebo Munyai

- Likely usage: Several times a week
- Main purpose: Sharing insights and knowledge
- Rating: 4 stars
- Ease of navigation: Easy
- Most useful features: Group discussions, Resource sharing
- Satisfaction with group discussions and private messages: Satisfied
- Comments: Likes the ease of navigation on the website.

### User 6: Thabiso Mofokeng

- Likely usage: Daily
- Main purpose: Accessing or sharing resources
- Rating: 4 stars
- Ease of navigation: Neutral
- Most useful features: Group discussions, Resource sharing
- Satisfaction with group discussions and private messages: Neutral
- Comments: Likes the ability to share resources.

### User II: Daniel

- Comments: Mentioned the application was fast and efficient. Suggested adding a walkthrough/tutorial for first-time users to explain buttons and pages.

### User III: Khanyisile Ndlovu

- Comments: Liked the notifications feature and linked course recommendations. Requested more customization options.

### User IV: Thato Shandu

- Comments: Found the application visually appealing. Suggested improving feedback forms to be more user-friendly and requested a holistic view of the full application.

### User V: Simphiwe Mathe

- Comments: Praised the stability of the application. Recommended adding an FAQ or help section for new users.

## 11.4 Summary of Suggested Improvements

Based on the collected feedback, several areas for improvement and additional features have been identified. Key observations include:

- Users appreciated the intuitive interface, responsiveness, and stability.
- There is a need for guidance for first-time users, including tutorials or walkthroughs.
- Feedback forms should be improved to be more user-friendly.
- Users requested more customization options for notifications and course recommendations.
- Adding an FAQ or help section would assist new users.

This feedback was collected by sharing the deployed application link with friends and colleagues, allowing them to interact with the live system and provide honest opinions.

## 11.5 Improvements made to accomodate users

- Implemented more features such as adding a friend recommendations, comment sections, group activities etc...
- Made some improvements with the look and feel for private chats.
- We aim to continue with adding more features and improvements for group discussions in the future sprint.
- We improved the websites response time for both mobile and computers to improve user experience and usability.
- Made changes to the user feedback forum to be more pleasing to the eye and intuitive to use.

## 11.6 Link to Google Form User Feedbacks

For reference and further responses, the Google Form can be accessed here:
`https://forms.gle/VkPyoACzZmBmYbd57`

# Chapter 12

# Database Documentation

## 12.1 Database Setup

### 12.1.1 Option A: Local PostgreSQL

1. Install PostgreSQL locally.
2. Create a database.
3. Update the `.env` file with your local credentials.

### 12.1.2 Option B: NeonDB (Recommended)

1. Sign up at `https://neon.tech`.
2. Create a new project.
3. Copy the connection string.
4. Update the `.env` file with NeonDB credentials.

## 12.2 Database Operations

```
# Run migrations
npm run db:migrate

# Seed database
npm run db:seed

# Reset database
npm run db:reset
```

Listing 12.1: Database Commands

## 12.3 Database Configuration

### 12.3.1 Sequelize Setup

The application uses Sequelize ORM with PostgreSQL for database interaction. Key configuration is stored in `config/database.js`:

```
const sequelize = new Sequelize(
  process.env.DB_NAME,
  process.env.DB_USER,
  process.env.DB_PASSWORD,
  {
    host: process.env.DB_HOST,
    port: process.env.DB_PORT,
    dialect: 'postgres',
    logging: process.env.NODE_ENV === 'development',
    dialectOptions: {
      ssl: {
        require: true,
        rejectUnauthorized: false
      }
    }
  }
);
```

Listing 12.2: Sequelize Configuration

### 12.3.2 Models

- **User:** Stores authentication and profile data.
- Easily extensible for additional models as required.

## 12.4 Development Workflow

### 12.4.1 Starting Development

```
# Install dependencies
npm install

# Set up environment
cp env.example .env
# Edit .env with your database credentials

# Start development server
npm run dev
```

### 12.4.2 Database Changes

```
# Enable database sync for development
ENABLE_DB_SYNC=true

# Use migrations for production
npm run db:migrate
```

### 12.4.3 Testing APIs

- Visit `http://localhost:3000/api-docs` for interactive documentation.
- Use Postman or curl for API testing.

## 12.5 Production Deployment

### 12.5.1 Environment Setup

```
NODE_ENV=production
ENABLE_DB_SYNC=false  # Never sync in production
```

### 12.5.2 Security Considerations

- Change all default secrets.
- Use strong JWT secrets.
- Enable HTTPS.
- Configure proper CORS origins.
- Set up rate limiting.

### 12.5.3 Database

- Use production-grade PostgreSQL.
- Set up automated backups.
- Configure connection pooling.
- Always use migrations instead of sync.

## 12.6 Troubleshooting

### 12.6.1 Common Issues

**Database Connection Failed:** Verify `.env` credentials and network connectivity.

**Port Already in Use:** Change the port or use `npx kill-port 3000`.

**Module Not Found:** Run `npm install` and check Node.js version.

**Database Sync Issues:** Set `ENABLE_DB_SYNC=true` for development; use migrations for production.

### 12.6.2 Logs

- Application logs appear in the console.
- Database queries are logged in development mode.
- Error details are displayed in development mode.

## 12.7 Scripts Reference

```
{
  "start": "node server.js",
  "dev": "nodemon server.js",
  "test": "jest",
  "db:migrate": "sequelize-cli db:migrate",
  "db:seed": "sequelize-cli db:seed:all",
  "db:reset": "sequelize-cli db:drop && sequelize-cli
   db:create && sequelize-cli db:migrate && sequelize-
   cli db:seed:all"
}
```

Listing 12.3: Package Scripts

## 12.8 Next Steps

- Add more models as required.
- Implement unit and integration tests.
- Introduce structured logging.
- Add health checks and monitoring.
- Set up a CI/CD pipeline for automated deployments.

## 12.9 Support

- Refer to the troubleshooting section for common issues.
- Review API documentation at `/api-docs`.
- Check console logs for error details.
- Ensure all environment variables are correctly set.

## 12.10 Database Schema

This section describes the schema used to store application data. Below is an example of the `notifications` table schema from our Neon database instance.

### 12.10.1 Notifications Table

| Column | Data Type | Constraints | Description |
|---|---|---|---|
| `id` | SERIAL | PRIMARY KEY | Unique identifier for each |
| `user_id` | UUID | NOT NULL | References the user receivi |
| `message` | VARCHAR(255) | NOT NULL | The content of the notifica |
| `title` | VARCHAR(255) | NOT NULL | Title or subject of the not |
| `read` | BOOLEAN | NOT NULL, DEFAULT FALSE | Indicates if the notificatio |
| `created_at` | TIMESTAMP WITH TIME ZONE | NOT NULL | Timestamp when the notif |

Table 12.1: Schema for the `notifications` table.

## 12.11 Choice Justification

In selecting a database solution for this project, Neon DB emerged as the most effective and forward-thinking option, surpassing traditional relational databases in flexibility, scalability, and performance. Neon offers a fully managed, serverless PostgreSQL platform, meaning it provides all the robust relational features of PostgreSQL while introducing modern cloud-native benefits such as autoscaling, high availability, and instant branching for development and testing.

Our project required a solution that could elegantly handle relational data, linking entities such as users, resources, and reports without compromising speed or reliability. Neon was the ideal fit because it allowed us to seamlessly model relationships between tables while benefiting from PostgreSQLs mature query capabilities. Unlike many other databases that either limit relational functionality (as seen in certain NoSQL systems) or impose heavy maintenance overhead (as with self-hosted relational databases), Neon combines ease of use with enterprise-grade reliability.

Additionally, Neons separation of compute and storage makes scaling cost-efficient and straightforward, ensuring that our application can evolve without disruptive migrations or manual scaling efforts. Its seamless integration with ORMs like Sequelize further streamlines development, allowing our team to maintain a clean codebase while interacting with the database intuitively. This strategic choice ensures that we can confidently build a future-proof application with high performance, robust data integrity, and operational simplicity.

## 12.12 High End Overview

- **User-Centric Relationships**:
  - The `User` table is central; most tables reference it via `user_id`, establishing clear ownership.
  - Tables like `Notifications`, `PrivateChats`, `Progress`, and `Study_sessions` are one-to-many with `User`.
- **Many-to-Many Relationships**:
  - `UserCourses` links `Users` and `Courses`.
  - `Group_members` links `Users` and `Study_groups`.
  - `Likes` links `Users` and `Resources`.
  - These join tables avoid data duplication but can grow large with many users or resources.
- **Self-Referencing Relationships**:
  - `Follows` and `Follows_requests` implement social connections between users.
  - Requires careful indexing to maintain query performance as the user count grows.
- **Resource-Centric Relationships**:
  - `Resources` connect to `Resource_tags`, `Resource_threads`, and `Resource_reports`.
  - High-traffic resources may create join hotspots.
- **Structure observation**:
  - **Normalisation**— our data is clearly normalised as a result of non redundant data each piece of information is stored only once (e.g., UserCourses links Users and Courses instead of repeating course info for every user).Separate tables for entities- Users, Courses, Resources, Likes, Groups, etc., each have their own table.
  - Foreign keys instead of repeated data Relationships are handled via foreign keys (user_id, course_id, etc.) rather than duplicating user or course details.

- **Scalability Considerations**:
  - Proper indexing on foreign keys is essential (`user_id`, `resource_id`, `course_id`, etc.).
  - Many-to-many relationships scale quadratically, with active users and resources.
  - Database scalability is addressed with indexing, query optimization, caching (e.g., Redis).

# Chapter 13

# API Integration

In this chapter, we discuss the integration of third-party Application Programming Interfaces (APIs) that enhance the functionality, scalability, and security of our system. Specifically, we focus on the use of the **Google Authentication API** and the **Firebase Authentication API**. These APIs were chosen due to their robust features, ease of integration, and ability to provide secure, reliable user authentication.

## 13.1 Google Authentication API

The Google Authentication API (Google Auth) enables users to log in using their existing Google accounts, reducing friction during registration and ensuring a seamless onboarding process. By leveraging OAuth 2.0 and OpenID Connect, Google Auth ensures:

- **Enhanced Security:** OAuth 2.0 provides token-based authentication, which eliminates the need to handle user passwords directly, reducing vulnerability to data breaches.

- **User Convenience:** Users can sign in without creating a new username and password, improving accessibility and reducing account management overhead.

- **Scalability:** The integration scales efficiently to accommodate a growing user base, as the authentication process is managed by Googles reliable infrastructure.

- **Cross-Platform Support:** Google Auth works across multiple devices and platforms, ensuring consistent login experiences for users.

## 13.2 Firebase Authentication API

Firebase Authentication complements Google Auth by offering a comprehensive, developer-friendly authentication solution that supports multiple sign-in methods, including email/password, phone number verification, and third-party identity providers.

- **Ease of Implementation:** Firebase provides Software Development Kits (SDKs) for popular programming languages and frameworks, reducing development complexity.
- **Real-Time Sync:** As part of the Firebase ecosystem, authentication status can be synchronized in real-time with other Firebase services, such as Firestore or Realtime Database.
- **Secure Token Management:** Firebase automatically handles token generation, renewal, and expiration, ensuring high levels of security.
- **Customizable User Experience:** Firebase offers customizable authentication flows to align with the branding and design of the application.

## 13.3 Justification of API Choices

The decision to integrate Google Auth and Firebase Authentication was driven by the need to provide a secure, scalable, and user-friendly authentication mechanism. Using these APIs offers the following advantages:

1. **Security Best Practices:** Both APIs use industry-standard encryption and authentication protocols, reducing risks associated with password storage.
2. **Reduced Development Overhead:** Leveraging these mature APIs significantly decreases the time and resources required to implement and maintain authentication features.
3. **Reliability and Scalability:** Both Google and Firebase are backed by Google Clouds infrastructure, ensuring minimal downtime and high scalability for future growth.
4. **Integration Flexibility:** These APIs integrate seamlessly with various frontend frameworks, backend services, and cloud environments, allowing for a flexible system design.

## 13.4 Event App API integration:

The team has been fortunate to find a group that will lend us their deployed api,regarding the events ,we plan to use their api in such a way that we can use their events that they created as a way in which Study buddy users can make an event that will correlate to study session.

1. **Concerns:** We can create an event according to their schema but we ran through the issue of how we will access it to display on our application.

2. **Discussed on parsing their api to backend:** The team came out with a viable strategy to use their API by asking for additional information on how to better use their api as asking them to change their api would be not ideal. `https://codexa-v1.github.io/Codexa/development/BackendApi.html`

## 13.5 Google Calender API:

The team has agreed to integrate study buddy with google calender API ,reason being is that students can mount studdy session events on it and it could be synced to any device as long as they have google.

1. **Concerns:** Pricing ,will we need to change some endpoints to make it a suitable add on.

## 13.6 Study Buddy API-scrutiny

This section evaluates the architecture, robustness, and scalability measures implemented in the Study Buddy API. It highlights the testing approach, schema design, scalability considerations, and deployment practices.

1. **Stress Testing:** The API undergoes regular stress and load testing to evaluate its performance under high-traffic scenarios. Postman Runner is used to simulate concurrent users and heavy request loads. Metrics such as response time, throughput, and error rate are measured to identify bottlenecks and improve system resilience.

2. **Schema Robustness:** The API follows a well-defined RESTful schema with clear API naming conventions according to endpoints use (e.g., `/users/123`). Proper use of HTTP status codes and versioning strategies further improves reliability and forward compatibility.

3. **Scalability Measures:** The API is designed to be stateless, meaning each request contains all necessary context (e.g., authentication via JWT). This enables horizontal scaling by distributing traffic across multiple servers using load balancers. Rate limiting and throttling mechanisms are in place to prevent abuse and maintain service quality.

4. **Deployed Site Practices:** The api is deployed on Render platform,as Render lets you provision PostgreSQL databases, run cron jobs, and set up background workers alongside your API all within one dashboard.

5. `https://sdp-project-zilb.onrender.com`

Overall, the integration of these APIs improves the systems authentication capabilities, enabling developers to focus more on core features while ensuring user data security and a smooth user experience.

# Chapter 14

# Wireframes

Wireframes are essential for visualizing the layout and structure of an application before implementation. The following figures illustrate the wireframe designs for the applications key screens.

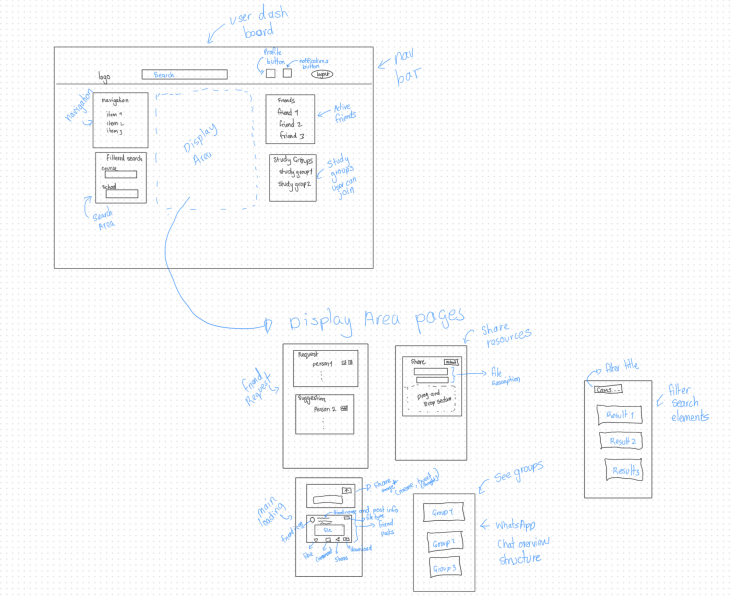



Figure 14.1: Dashboard UI

## Discussion

These wireframes provide a blueprint for the user interface, ensuring consistency and usability throughout the application.

# Chapter 15

# Features

- **Power feat:Resources**
  - Can be able to add resources pdfs and images.
  - Resources can be liked as well as commented on adding the feature of other users to have more intel if a certain Resource has been beneficial to their studying.
  - Can be able to search resources by title.
  - Can be able to download resources and save them locally.
  - Implemented this feature as resources being shared are important as they help students to study materials that are related to their course without wasting time looking at a myriad of things that are not related at all to their current study period
  - Resources that are shared by friends are what can be seen so we do not expect users to see all the resources that are uploaded into the application .Only people that are accepted as friend requests are the only peoples resources that you can see.
- **Find Study buddies**
  - View profiles of people using the study Buddy application by clicking their names on dashboard.
  - Can be able to see all the interests of the other student,as you view their profile.
  - Makes suggestions and can be able to accept friends request from people using the application.
  - With wanting to make a friend ,users can go to the study buddy navigation and can press the add add friend so that people could be added as friends .Then the above functionalities will apply as friend requests have been established.

- **Create Study Groups**
  - Create,leave and join group study groups .
  - Within my groups functionality you can see the groups you are in as well as have the ability to create sessions where by time and location can be inserted so that members know the specific details of these.
  - Implemented real-time chat system where clicking a persons profile ensures the chat system appears and can be ready for use .
  - Chat system has real-time message retrieval system.
- **Track Your Progress**
  - Calendar event planner ,when click on date it will show you the reminder and time to finish that task
  - Log any completed topics, subtopics, and sections .
  - Track and view hours spent studying.
  - Can be able to see study sessions that were planned according to the study groups that you are in.
- **Plan Sessions**
  - Notifications are done theres a notifications page that appears with all the notifications about tasks .
  - Add automatic reminder through email – in progress
  - Schedule future study sessions with reminders within study groups .

# Chapter 16

# Stakeholder

- **Email sign in**
  - Client proposed that we add a feature to sign in to the application through email and password ,adding upon the gmail(google sign in).
  - Done - implemented this through firebase as we dont want to be responsible for storing personal user information like passwords and emails as its risky.
  - **Status** – Done.
- **Google Calendar API**
  - Client got engaged and recommended the integration of google calendar api as they saw how it could be well suited to store study events for students.
  - Cost was the concern our client highlighted as something to lookout for.
  - **Status** – Done.
- **Comment section under resources**
  - Client found this fascinating and liked the idea behind this feature as well as the liking of resources.
  - **Status** – Done.
- **Enhancing the Friend request UI**
  - Client adviced the team to add better ui elements to the friends list instead of the blank letters of the user name hence improving usability and improving aesthetic feel.
  - **Status** – Done.
- **feat: filter resources by friends instead of all**

  - The Client mentioned the fact that users should see resources that are posted by their friends instead of making it broad and seeing all resources as it could overwhelm the student.
  - **Status** – In progress.

- **Group features**
  - Client liked the idea of having groups that he can join freely and interact with the people who are also the member of the group.
  - **Status** – Done.

# Chapter 17

# Performance

## 17.1 Google Lighthouse

Google Lighthouse is an open-source, automated tool developed by Google to improve the quality of web pages. It provides audits for performance, accessibility, best practices, SEO, and progressive web apps. By running Lighthouse, developers receive detailed reports with actionable recommendations to optimize their websites. The main benefits include faster page load times, improved user experience, better accessibility compliance, enhanced search engine visibility, and adherence to modern web standards. Lighthouse can be run in Chrome DevTools, from the command line, or integrated into continuous integration pipelines to ensure consistent web performance improvements.

## 17.2 Mobile

**Metrics:**

- First Contentful Paint - 1.7s
- Total Blocking Time - 20ms
- Speed Index - 1.7s
- Largest Contentful Paint - 4.4s
- Cumulative Layout Shift - 0.111

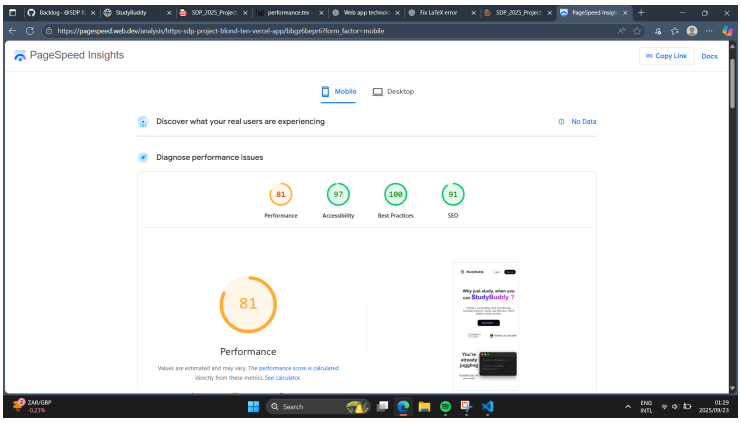



Figure 17.1: How Study Buddy performs on mobile devices

## 17.3 Desktop

**Metrics:**

- First Contentful Paint - 0.4s
- Total Blocking Time - 0ms
- Speed Index - 0.4s
- Largest Contentful Paint - 0.8s
- Cumulative Layout Shift - 0.118

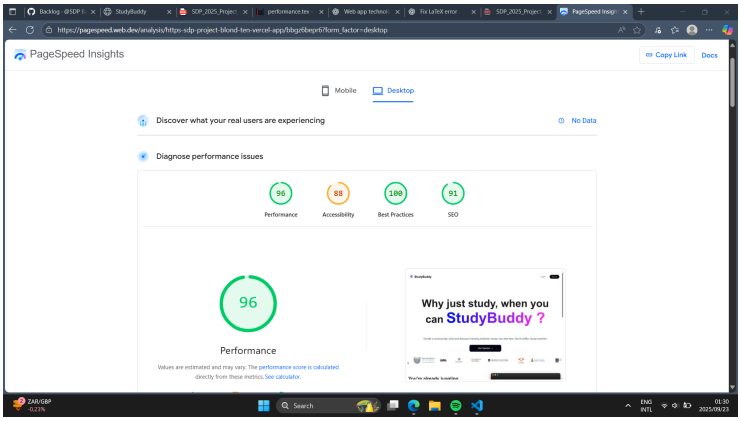



Figure 17.2: How Study Buddy performs on desktops